ЛЦТ 2026 · СПЕЦИАЛЬНЫЙ ТРЕК ФСП

ФСП Карьера

Цифровая платформа подбора ИТ-специалистов с обратной механикой и верифицированным профилем достижений участника Федерации спортивного программирования.

Сопроводительная документация

Демо: https://brokenbrain.online · OpenAPI: /openapi.yaml · Репозиторий: github.com/06ED/fsp-career

Команда: ЛЦТ 2026, спецтрек ФСП

Содержание

- Обзор решения
  - Ключевые решения
  - Демонстрация
  - Соответствие требованиям ТЗ
- Архитектура решения
  - Функциональная архитектура
  - Компонентная архитектура
  - Модули API
  - Данные
- Механика тестирования
  - Путь кандидата
  - Почему так: две проблемы из ТЗ
  - Банк заданий
  - Адаптивный тест (CAT)
  - Правила грейда и пересдачи
  - Задания из описания вакансии
  - История и связь с подбором
  - Регулярные короткие задания
- Механика подбора и категоризация
  - Сценарий выхода на контакт
  - Категоризация
  - Два режима выдачи
  - Формула релевантности
  - Объяснимость
  - Приватность в выдаче
  - Дополнительный сценарий: вакансии и отклики
- Процедура валидации и результаты
  - Тестирование
  - Подбор
  - Функциональная проверка
  - Ограничения и план валидации на реальных данных
- Интеграция с реестром ФСП
- Безопасность и 152-ФЗ
- Защита от фиктивных вакансий и недобросовестных работодателей (концепция)
- Интеграция с ATS работодателей (концепция)
- Описание API
  - Служебные
  - Аккаунт и роль
  - Профиль кандидата, резюме, приватность, ФСП
  - Тестирование
  - Работодатель: компания и потребность
  - Подбор и банк кандидатов

- Приглашения
- Вакансии и отклики
- Регулярные задания

Библиотеки и компоненты

- Инфраструктура
- Backend (`apps/api`)
- Frontend (`apps/web`)
- Шрифты и бренд

Сборка, развёртывание и локальный запуск

- Развёрнутый прототип
- Локальный запуск (Docker)
- Сборка без Docker (разработка)
- Промышленное развёртывание (как устроено на brokenbrain.online)

Ограничения и план развития

- Ограничения прототипа
- План развития

Обзор решения

ФСП Карьера — веб-платформа подбора ИТ-специалистов с обратной механикой. Специалист не рассылает резюме. Он один раз проходит обязательный путь (опрос по отрасли и специализации → выбор грейда → адаптивный тест на этот грейд) и получает подтверждённую категорию «специализация · грейд». Работодатель описывает потребность команды и получает ранжированную, объяснимую подборку из категоризированной базы. Затем он сам отправляет приглашение конкретному человеку с описанием предложения и зарплатной вилкой. Контакты кандидата открываются только после его согласия. Профиль обогащается подтверждёнными достижениями из реестра ФСП: привязка ФСП ID через Keycloak-совместимый контур.

Ключевые решения

- **Тестирование, устойчивое к утечке заданий.** Задания собираются из семейств параметрических и изоморфных заданий с откалиброванной сложностью (IRT 2PL). Каждый кандидат получает уникальные экземпляры сопоставимой трудности. Тест адаптивный и строится на заявленный грейд, а приоритет тем можно задать описанием вакансии.
- **Объяснимое ранжирование.** Подтверждённый тест, близость текста опыта к описанию потребности (TF-IDF + русский стеммер), покрытие стека с поправкой на достоверность, достижения ФСП, отрасль и актуальность. У каждой позиции в выдаче есть разбор по факторам.
- **Собственная валидация с результатами.** Синтетические популяции и рынки с эталонной разметкой, базовые линии, бутстрэп-доверительные интервалы, абляции и симуляция атаки «слитым банком». Отчёты лежат в `docs/validation/` и воспроизводятся командами Gradle.
- **Совместимость с ФСП ID.** Аутентификация через Keycloak 26. Привязка участника ФСП подтверждается владением учётной записью, а достижения импортируются из реестра через сменный коннектор.

Демонстрация

Что	Где
Платформа	https://brokenbrain.online
Демо-почта (письма подтверждения регистрации)	https://brokenbrain.online/mail/
OpenAPI	https://brokenbrain.online/openapi.yaml
Репозиторий	https://github.com/06ED/fsp-career

Демо-логины (пароль `fsp-demo-2026`; все данные синтетические):

Логин	Роль	Что показать
`employer@demo.fsp.local`	Работодатель «Демо Финтех»	Готовая потребность, подборка с объяснением, банк кандидатов, приглашения во всех статусах (у принятого видны контакты), вакансии с откликами, задания и ответы
`maria.volkova@demo.fsp.local`	Специалист, протестирована (backend · middle)	Входящее приглашение с вилкой и способом связи, принять или отклонить; привязанный ФСП ID с достижениями
`ivan.petrov@demo.fsp.local`	Новый пользователь	Сквозной путь: выбор роли → профиль (или импорт PDF) → опрос → тест на заявленный грейд → категория → привязка `FSP-100001` → вакансии и задания

Можно и зарегистрироваться с любым адресом. Письмо с подтверждением придёт в демо-почту, если на стенде не настроен SMTP.

Сквозной сценарий (как в п. 3.3 ТЗ).

1. Регистрация специалиста.
2. Опрос и тест.
3. Присвоение категории.
4. Работодатель задаёт потребность и находит категорию.
5. Приглашение конкретному человеку.
6. Принятие и раскрытие контактов.
7. Отдельно — отклик на вакансию.

Соответствие требованиям ТЗ

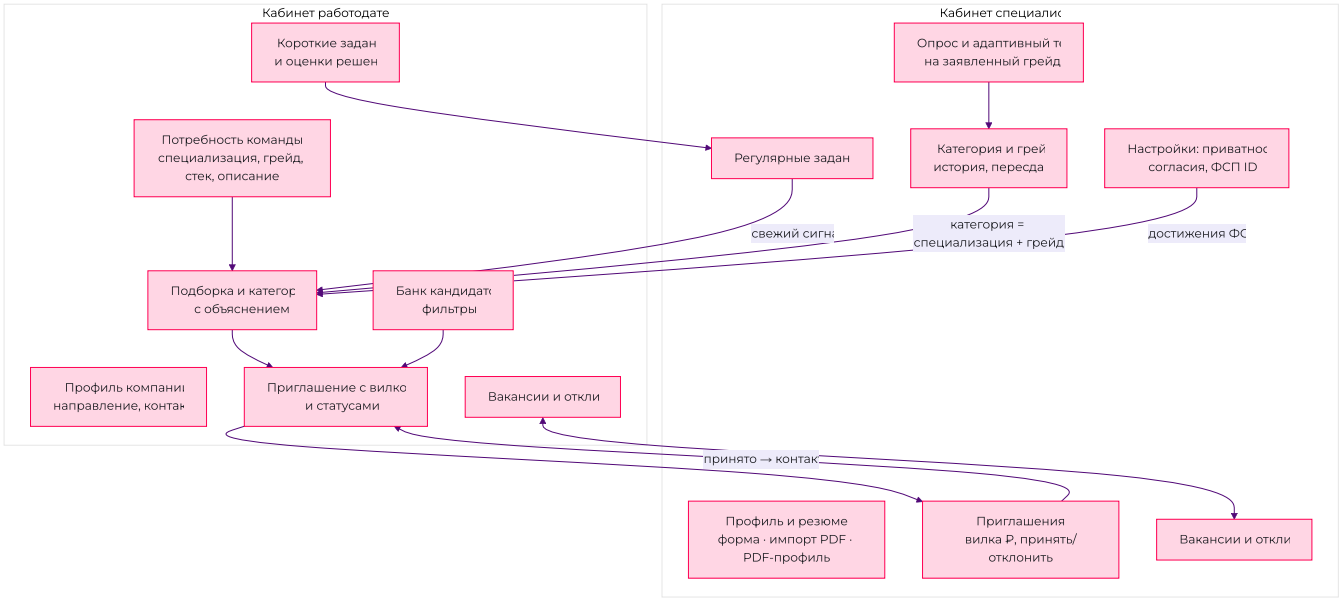
Требование	Реализация
Профиль и резюме: форма или PDF с авторазпознаванием, стандартизированный PDF-профиль	Форма профиля. Импорт PDF (PDFBox) с распознаванием ФИО, контактов, стека и опыта в черновик с проверкой кандидатом. Генерация брендированного PDF-профиля
Тестирование: опрос, выбор грейда, тест, категория	Опрос (отрасль, специализация, стаж, роли, формат) → заявленный грейд → адаптивный тест → категория, видимая работодателю
Грейд не понижается; тест на уровень ниже или выше; ограничение по времени	Подтверждённый грейд — максимум из достигнутых. После неудачи можно сразу пройти тест на уровень ниже. Повторный тест на тот же или более высокий уровень — через 90 дней

Требование	Реализация
Регулярные задания (желательно)	Еженедельное задание от работодателя (детерминированная ротация), решение или подход, оценка 1–5. Активность учитывается в ранжировании
Категория и грейд, история опроса и теста	Текущий статус и история попыток с опросом, целью, результатом и датами
Приглашения: вакансия или предложение, вилка, принять или отклонить	Вилка «от–до» ₽ обязательна (CHECK в БД), способ связи, статусы отправлено / просмотрено / принято / отклонено
Вакансии и отклик (желательно)	Лента вакансий с фильтрами, отклик, статусы, отзыв отклика
Настройки: приватность, согласия, ФСП ID	Раздельные согласия, приватность по разделам, привязка ФСП ID с проверкой владения
Ограничение доступа кандидата	Только свои приглашения и отклики, проверка на сервере
Профиль компании	Название, описание, направление деятельности, контакты
Описание потребности	Специализация, грейд, отрасль, стек, описание команды
Подборка: категории, кандидаты, обоснование	Рекомендуемые категории, ранжированная подборка, разбор факторов, карточка
Поиск по банку с фильтрами	Специализация, грейд, категория, стек, достижения ФСП, отрасль, текст
Приглашение без привязки к вакансии; статусы	Да
Публикация вакансий (желательно)	Создание, редактирование, закрытие, работа с откликами
Контакты скрыты до принятия или собственного отклика	Единое правило `disclosedCandidates()`, проверено HTTP-тестами
Ранжирование внутри категории по силе подтверждённого профиля; по релевантности потребности с объяснением	Режимы «банк» и «подборка»
Регистрация по e-mail с подтверждением	Keycloak: подтверждение обязательно
Корректная обработка кандидата без истории ФСП	Фактор 0, без штрафа, явный текст, юнит-тест
Механизм защиты от фиктивных вакансий, интеграция с ATS (плюс)	Концепции — разделы «Защита...» и «Интеграция с ATS»
OpenAPI, Docker, Keycloak, открытый код	`openapi.yaml`, Dockerfile и Compose, Keycloak 26, лицензия MIT
Адаптивная вёрстка (опционально)	Медиа-запросы для телефона и планшета

Требование	Реализация
ML/NLP (преимущество)	TF-IDF + стеммер для подбора; IRT-оценка способности; распознавание PDF

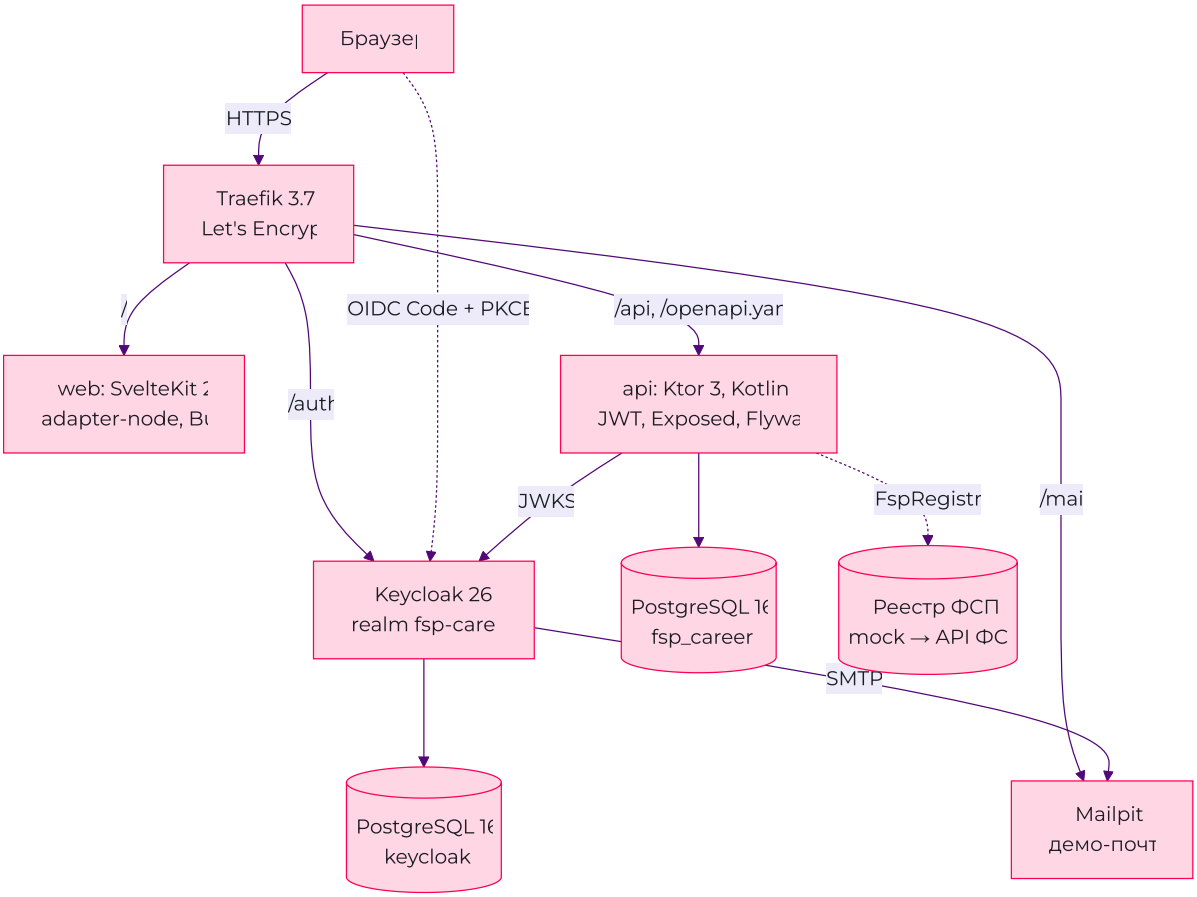
Архитектура решения

Функциональная архитектура



Главный сценарий — обратный подбор. Специалист не откликается вслепую. Он один раз проходит обязательный путь (опрос → выбор грейда → тест) и получает подтверждённую категорию. Работодатель описывает потребность, получает ранжированную подборку с объяснением и сам отправляет приглашение с описанием и зарплатной вилкой. Контакты специалиста открываются только после того, как он примет приглашение. Классический сценарий (вакансия и отклик) реализован как дополнительный.

Компонентная архитектура



Слой	Компонент	Ответственность
Frontend	`apps/web` (SvelteKit, TypeScript, StyleX)	Лендинг и единое веб-приложение с двумя кабинетами. Вход через keycloak-js, вызовы API с Bearer-токеном.
Backend	`apps/api` (Ktor, монолит с модулями)	Вся бизнес-логика и авторизация на сервере, OpenAPI-спецификация.
Хранение	PostgreSQL 16	Реляционная схема под управлением Flyway (миграции `V1…V5`), JSONB для списков (стек, навыки).
Идентификация	Keycloak 26	Регистрация по e-mail с подтверждением, вход и сессии. Подготовлен к подключению ФСП ID как Identity Provider.
Инфраструктура	Docker Compose, Traefik, GitHub Actions	Воспроизводимый запуск, HTTPS, автодеплой.

Модули API

Монолит разделён на слабо связанные модули. Каждый владеет своими таблицами, моделями и маршрутами и общается с другими через узкие функции хранилища.

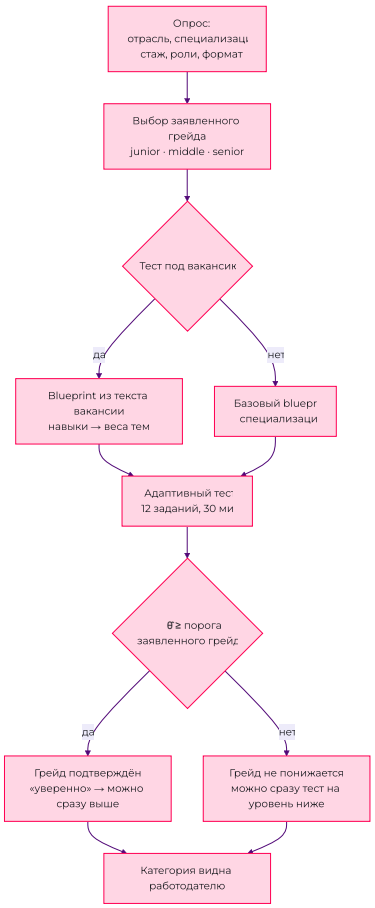
Модуль	Файлы	Что делает
Аккаунты и роли	`AccountRoutes.kt`, `CareerStore.kt`	`GET /api/me`, одноразовый выбор роли `POST /api/me/role`, проверка роли и владения записями.
Профиль и резюме	`CareerRoutes.kt`, `ResumeRoutes.kt`, `PdfExport.kt`	Профиль, согласия, импорт PDF с автораспознаванием в черновик, генерация PDF-профиля.
Тестирование	`assessment/*`	Справочники, банк семейств заданий, генераторы, адаптивный тест, правила пересдачи, построение теста по вакансии.
Подбор	`matching/*`	Признаки, ранжирование, объяснения, рекомендуемые категории, банк кандидатов.
ФСП	`FspRegistry`, `MockFspRegistry`	Привязка ФСП ID, импорт достижений, политика баллов.
Приглашения	`CareerRoutes.kt`, `CareerStore.kt`	Приглашения с вилкой ₽ и статусами, раскрытие контактов после принятия.
Вакансии и отклики	`Vacancy*.kt`	Публикация, отклики, статусы, раскрытие контактов при отклике.
Регулярные задания	`Task*.kt`	Еженедельная ротация заданий, решения, оценки работодателя, сигнал актуальности.

Данные

Основные сущности: `accounts` (subject Keycloak → роль); `candidate_profiles` (профиль, согласия, приватность, ФСП ID); `employer_companies` и `employer_needs`; `assessment_attempts` (опрос, заявленный грейд, θ, результат, подтверждённый грейд) и ответы; `achievements` (подтверждённые достижения ФСП); `invitations` (описание, вилка ₽, способ связи, статус); `vacancies`, `vacancy_applications`; `short_tasks`, `short_task_submissions`, `short_task_offers`. Ограничения целостности (CHECK, UNIQUE, FK с каскадом) закреплены в миграциях: например, вилка зарплаты `min > 0` и `max ≥ min`, а статусы перечислены явно.

Механика тестирования

Путь кандидата



Почему так: две проблемы из ТЗ

Проблема	Решение
Единый набор заданий быстро утекает	Банк из **семейств заданий**. Семейство — это один навык на одной откалиброванной сложности. 59 из 76 семейств — **параметрические генераторы**: каждый кандидат получает свой экземпляр (новые числа, таблица, фрагмент кода), а сервер пересчитывает правильный ответ из seed и хранит его только у себя. Остальные 17 — наборы изоморфных вариантов. Выученный ответ чужого экземпляра бесполезен.
Генерация нейросетью не гарантирует сопоставимую сложность	Все экземпляры семейства имеют **одинаковые параметры IRT** (2PL: дискриминативность *a*, трудность *b*, у заданий с выбором — асимптота угадывания *c* = 1/число вариантов). Поэтому формы сопоставимы, а оценка объективна: это шкала способности θ, а не доля верных ответов на случайный набор.

Банк заданий

76 семейств: backend 26, frontend 28, данные 27, продукт 15, дизайн 19 (в каждую специализацию входят 11 общих семейств CS). Примеры генераторов: вывод фрагмента кода на Kotlin, Python или JS со случайными значениями; сложность сгенерированного цикла; результат SQL-запроса по сгенерированной таблице; HTTP-статус для сценария; уровень изоляции транзакций; специфичность CSS; порядок event loop; рендер и эффекты React; статистика (медиана, перцентиль, конверсия, значимость); продуктовые метрики (retention, ARPU, воронка); контраст и доступность. Задания на русском. Ключи ответов никогда не отдаются клиенту, а корректность во время теста не раскрывается.

Адаптивный тест (CAT)

- **Старт** в якоре заявленного грейда на шкале θ: junior −1.0, middle 0.15, senior 1.4.
- **Выбор следующего задания**: максимум информации Фишера при текущей оценке θ в пределах квот тем из blueprint. Среди почти равных кандидатов (≥ 85% информации лучшего) берётся наименее показанное семейство — это контроль экспозиции.
- **Оценка**: EAP с популяционным априорным N(0, 1). Заявленный грейд влияет только на маршрут теста, но не на итоговую оценку. Балл 0–100 = 100·Φ(θ̂).
- **Пороги грейдов**: middle ≥ −0.5, senior ≥ 0.8 (примерно 31 и 79 баллов). Заявленный грейд подтверждён, если θ̂ − k·SE ≥ порога. k = 0 выбран по минимуму потерь в валидационном переборе: завышение весит вдвое больше занижения. «Уверенно» означает, что θ̂ выше порога следующего грейда.
- 12 заданий, 30 минут, одна открытая попытка. Брошенная или просроченная попытка засчитывается как неудачная, а неотвеченные задания считаются неверными.

Правила грейда и пересдачи

- **Грейд не понижается**: подтверждённый грейд = max(прежний, заявленный при успехе). Первая неудача даёт junior как начальный уровень.
- **Повтор** на тот же или более высокий уровень — не раньше чем через **90 дней** (по специализации).
- **Сразу можно**: после неудачи — один тест на уровень ниже; после уверенного успеха — один тест на уровень выше, если этот уровень не был провален в окне 90 дней.
- Нарушение правил возвращает `409` с причиной и датой `retakeAvailableAt`. Интерфейс показывает, какие грейды доступны сейчас.

Задания из описания вакансии

Функция `blueprintFor(специализация, текст вакансии)` выделяет навыки из текста по русско-английскому словарю с синонимами и лёгким стеммингом. Найденные навыки превращаются в веса тем: 60% от вакансии, 40% — ядро специализации. Веса дают квоты заданий. В приложении есть раздел «Как формируются задания»: демо-вакансия → найденные навыки → план тем → пример сгенерированных заданий (без ответов). Есть пять демо-вакансий: Kotlin-бэкенд в финтехе, React/TS-маркетплейс, аналитик в EdTech, PM подписочного сервиса, дизайнер банка. Кандидат может пройти тест «под вакансию».

История и связь с подбором

История хранит опрос, заявленный грейд, вакансию, θ ± SE, число верных ответов, итог и грейд до и после. Темы, подтверждённые в тестах (не меньше двух верных ответов и не меньше 2/3 верных по теме), передаются в подбор как **подтверждённые навыки**: в факторе стека они весят 1.0, а навык, только заявленный в профиле, — до 0.5.

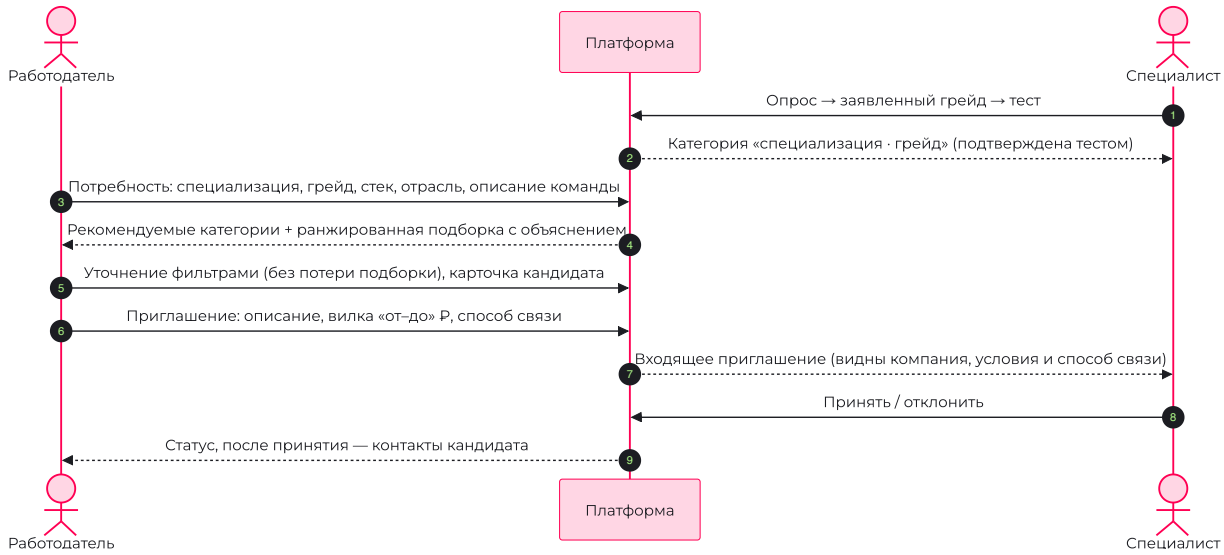
Регулярные короткие задания

Раз в неделю система предлагает кандидату задание от работодателя его специализации и грейда. Выбор детерминированный: SHA-256 от «кандидат | неделя» по модулю пула. Поэтому в одну неделю у разных кандидатов разные задания. Ответить можно решением или описанием подхода. Работодатель видит ответы (категорию, но не контакты) и ставит оценку 1–5. Ответ обновляет активность кандидата, а она учитывается в факторе актуальности ранжирования.

Механика подбора и категоризация

Сценарий выхода на контакт

Инициативу проявляет работодатель, как в модели LHH: он не разбирает поток откликов, а работает с уже категоризированной базой.



Чем это отличается от обычных площадок.

1. Выдача строится по категории, подтверждённой тестом, а не по самоописанию.
2. Кандидат видит условия, включая зарплатную вилку, до начала общения.
3. Контакты защищены до явного согласия кандидата.
4. Каждая позиция в выдаче объяснена.

Категоризация

Категория — это пара «специализация · подтверждённый грейд», например «Backend-разработка · Middle». Справочник специализаций: backend, frontend, данные и аналитика, продукт, дизайн. Грейды: junior, middle, senior. Категория появляется только после теста. Заявленный грейд, грейд в резюме и список навыков не могут ни создать категорию, ни повысить её. Учитывается лучший результат в текущей специализации.

Два режима выдачи

Режим	Где	Порядок
Подборка под потребность	«Подбор», `GET /api/matches`	Релевантность потребности работодателя (формула ниже)
Банк кандидатов / категория	«Банк», `GET /api/bank/candidates`	Сила подтверждённого профиля: `65·тест + 25·ФСП + 10·актуальность`

Рекомендуемые категории — это категории с количеством подходящих кандидатов и их средней силой профиля. Клик по категории открывает её в банке. Фильтры: специализация, грейд, категория, стек (все указанные навыки, с учётом синонимов), наличие достижений ФСП, отрасль, свободный текст. Фильтры уточняют уже полученную выборку на клиенте, и она не теряется. Переключение между режимами сохраняет обе выборки.

Формула релевантности

Это чистая функция `matching/MatchingEngine.kt`, формула описана в комментарии к ней.

1. **Фильтр по категории.** Проходят только опубликованные кандидаты (дано согласие на обработку и на публикацию) с подтверждённой специализацией, совпадающей с нужной. Множитель за грейд: точное совпадение ×1.00, на ступень выше ×0.85, на две ×0.70. Грейд ниже требуемого исключается; его можно включить флагом `allowLowerGrade`, тогда множитель ×0.60.
2. **Релевантность** = множитель грейда × сумма факторов (из 100):

Фактор	Вес	Как считается
Тест	30	нормированный балл теста (0–100)
Текст	25	TF-IDF-косинус между описанием потребности (плюс стек ×2) и описанием опыта кандидата, насыщение при 0.35
Стек	20	по каждому требуемому навыку: подтверждён тестом 1.0; указан и описан в опыте 0.8; только описан 0.5; только указан 0.5 × r, где r = (подкреплённые + 1)/(указанные + 2) — достоверность всего заявленного списка
ФСП	15	сумма баллов подтверждённых достижений с учётом давности (1.0 первые два года, затем −0.2 в год, минимум 0.4) / 20, не больше 1
Отрасль	5	совпадает — 1.0, упомянута в опыте — 0.5
Актуальность	5	1.0, если активность была не раньше 30 дней назад, линейно до 0.3 к году. Активность — это обновление профиля, тест, привязка ФСП или ответ на регулярное задание

Если в потребности нет стека или отрасли, соответствующий фактор исключается, а остальные масштабируются до 100. При равенстве порядок определяется силой профиля, затем ID кандидата, поэтому он детерминирован.

NLP. Текст токенизируется (русский и английский), русские слова приводятся к основе стеммером Snowball, а ИТ-термины канонизируются: «постгрес», «PostgreSQL» и «postgres» считаются одним термином. Каждый документ векторизуется TF-IDF по всему пулу. Теги стека не входят в текстовый документ, чтобы не учитываться дважды. Благодаря этому «накрутка» стека тегами без подкрепления опытом почти ничего не даёт.

Баллы ФСП вычисляются по версионируемой политике: база по уровню (международный 10, всероссийский 7, региональный 4) × исход (победитель 1.6, призёр 1.3, финалист 1.0, участник 0.5), от 1 до 20 за результат. **Кандидат без истории ФСП** получает за этот фактор 0, и больше ничего не меняется (это закреплено юнит-тестом). В объяснении выводится строка «История ФСП не привязана — фактор не начислен (это не штраф)».

Объяснимость

У каждой позиции есть `factors[]`: вклад фактора в баллах и русская формулировка. Например: «Тест: 82/100, подтверждённый грейд middle», «Стек: 3 из 4 — Kotlin, PostgreSQL, Docker; нет: Kafka», «Достижения ФСП: призёр финала…», «Отрасль совпадает: финтех». Кроме того, есть флаги («Грейд выше требуемого», «Нет истории ФСП», «Профиль давно не обновлялся») и краткое резюме. В интерфейсе факторы показаны полосками, а карточка кандидата объясняет, почему он попал в подборку.

Приватность в выдаче

Кандидат выбирает, что видно работодателю: ФИО (иначе «Кандидат №…»), стек, опыт, историю работы, гибкие навыки, достижения ФСП. Скрытые разделы не возвращаются API и **не используются в ранжировании**. E-mail и телефон никогда не попадают в выдачу. Они раскрываются в карточке и в приглашениях, только если кандидат принял приглашение этого работодателя или сам откликнулся на его вакансию. Это правило реализовано в одной функции `disclosedCandidates()`.

Дополнительный сценарий: вакансии и отклики

Работодатель публикует вакансии (вилка ₽ обязательна, формат работы, город), редактирует и закрывает их и работает с откликами со статусами «отправлен → просмотрен → принят / отклонён». Кандидат видит ленту вакансий, где сначала идут подходящие его категории, с объяснением совпадения. Откликнуться можно, только если подтверждена категория: работодатель должен видеть результат теста. Отклик сразу раскрывает контакты этому работодателю, а его отзыв удаляет раскрытие.

Процедура валидации и результаты

Готовых датасетов и эталонной разметки ТЗ не даёт, поэтому качество механик проверяется на **синтетических данных с известной истиной**. Генератор задаёт скрытые атрибуты: истинный уровень, навыки, отрасль. Платформа видит только зашумлённые наблюдения, как в продукте. Эталон считается только по скрытым атрибутам, так что система не может подсмотреть ответ. Все прогоны детерминированы и воспроизводятся одной командой:

```
cd apps/api
gradle validateAssessment   # → docs/validation/assessment-results.{md,json}
gradle validateMatching     # → docs/validation/matching-results.{md,json}, matching-
        pairs.csv
```

Полные отчёты с методикой, таблицами по специализациям и потребностям, абляциями и чувствительностью лежат в `docs/validation/`. Ниже — сводка.

Тестирование

Модель. 10 000 синтетических кандидатов (по 2 000 на специализацию), истинный уровень $\theta \sim N(0, 1)$. Эталонный грейд — полоса θ по тем же порогам. Самооценка кандидата зашумлена и в среднем завышена. Ответы генерируются по 3PL с параметрами семейства, но трудность каждого экземпляра смещена на $\delta \sim N(0; 0.25^2)$, то есть изоморфность неидеальна. Движок о δ не знает. Каждый кандидат проходит весь путь дважды на независимых экземплярах. Симуляция вызывает те же объекты, что и API: генераторы, blueprint, CAT, EAP, правила грейда и пересдачи. Для сравнения тот же набор кандидатов прогнан через прежний механизм: 5 случайных вопросов из банка в 25 штук, грейд по доле верных.

Критерий жюри	Метрика	Новый механизм	Прежний
Соответствие эталону	точность грейда	**78.6%**	27.2%
Соответствие эталону	macro-F1	**0.78**	0.23
Корректность категоризации	доля завышенных грейдов	**6.5%**	71.8%
Согласованность	взвешенная каппа при повторе на других экземплярах	**0.71**	0.17
Согласованность	корреляция оценок двух прохождений	**0.77**	0.23
Дискриминативность	корреляция оценки с истинным θ	**0.87**	0.46

Критерий жюри	Метрика	Новый механизм	Прежний
Дискриминативность	средняя item–rest корреляция семейств	**0.37**	—
Устойчивость к утечкам	доля идентичных заданий у двух кандидатов	**1.9%**	68.1%
Устойчивость к утечкам	эффект от выученной утечки 1 000 показанных вопросов	**+5.7 балла** (завышение 7.5% → 13.9%)	весь банк: 100 баллов и senior у всех

Надёжность теста — 0.76, средняя SE — 0.48. Одно семейство с item–rest ниже 0.20 помечено на доработку. **Слабые места** отчёт показывает честно: наибольший выигрыш от утечки дают концептуальные семейства с вариантами, особенно в дизайне (+8.4 балла при утечке 1 000) и продукте (+7.0). Их расширяем и ротируем в первую очередь. Порог k = 0, длина теста 12 и правило выбора заданий выбраны перебором; результаты устойчивы к разбросу трудности экземпляров от 0 до 0.5.

Подбор

Тестовый набор пар «вакансия – кандидат». Пять независимых синтетических рынков по 360 кандидатов и 24 потребности работодателей, написанные как русские описания вакансий. Пять специализаций, три грейда, девять отраслей. Наблюдения зашумлены: тест ошибается на ступень с вероятностью 0.15; 30% кандидатов добавляют в стек 1–2 выдуманных навыка, 20% — «накрутчики» с 4–6 навыками; 25% завышают самооценку грейда; история ФСП есть лишь у части кандидатов. Эталонная релевантность 0–3 считается по скрытой истине (уровень, покрытие стека, отрасль). Её формула **намеренно отличается** от формулы движка. Метрики P@k, nDCG@10 и MRR усреднены по 120 парам «потребность × рынок», 95% доверительные интервалы — бутстрэпом. Веса задавались заранее, без подгонки. Набор пар с метками лежит в `docs/validation/matching-pairs.csv`, так что жюри может перепроверить.

Система	P@5	P@10	nDCG@10	MRR
Подбор под потребность (платформа)	**0.74**	**0.62**	**0.709** [0.685–0.731]	**0.919**
Категория + сила профиля (режим банка)	0.65	0.57	0.612	0.840
Только балл теста	0.55	0.50	0.549	0.711
Ключевые слова + заявленный стек («как на обычных площадках»)	0.41	0.37	0.433	0.648

Система	P@5	P@10	nDCG@10	MRR
Случайный порядок внутри категории	0.42	0.42	0.447	0.623
Случайный порядок по специализации	0.19	0.19	0.216	0.390

- Движок превосходит поиск по ключевым словам на **+0.276 nDCG@10** [0.246; 0.306].
- Учёт описания потребности (текст, стек, отрасль) даёт +0.13 nDCG@10, а отказ от результатов теста стоит −0.06.
- «Накрутчики» стека занимают 51% топ-10 у поиска по ключевым словам и 23% у движка при доле в пуле 20%.
- Изменение любого веса ×0.5 или ×1.5 оставляет nDCG@10 в диапазоне 0.636–0.732: ранжирование не держится на тонкой настройке.
- Исключение фактора ФСП на синтетике чуть улучшает метрику (+0.013). Генератор считает ФСП слабым сигналом, а вес оставлен, потому что ТЗ требует поднимать подтверждённые достижения. Это отражено в отчёте.

Функциональная проверка

- **Автотесты API:** 96 тестов (`gradle test`), включая HTTP-сценарии на настоящем Ktor-модуле с подписанными RSA-токенами через JWKS. Покрыты:
  - выбор роли;
  - тест (провал senior → доступен только middle → успех → блокировки → истечение);
  - подбор, банк, карточка, приватность, привязка ФСП;
  - вакансии, отклики и раскрытие контактов, еженедельные задания;
  - коды 400, 401, 403, 404 и 409.
- **Миграции** проверяются на PostgreSQL 16, в том числе поверх базы с данными (V1 → V7).
- **Smoke-проверки:** `docs/smoke_*.py` для локального стенда; `docs/smoke_demo.py` для развёрнутого стенда (вход через форму Keycloak и PKCE, подборка с объяснениями, правило раскрытия контактов, приглашение кандидату).
- **Фронтенд:** `svelte-check` без ошибок, production-сборка, Prettier.

Ограничения и план валидации на реальных данных

Результаты получены на синтетике с экспертными параметрами. Это проверка того, что механика восстанавливает заданную истину по зашумлённым сигналам, а не доказательство качества найма. План:

1. **Пилот теста.** Не меньше 200–300 ответов на семейство; калибровка 2PL/3PL (marginal ML), проверка согласия, одномерности и DIF по отраслям. Модель со случайным эффектом экземпляра, чтобы измерить реальную неизоморфность.

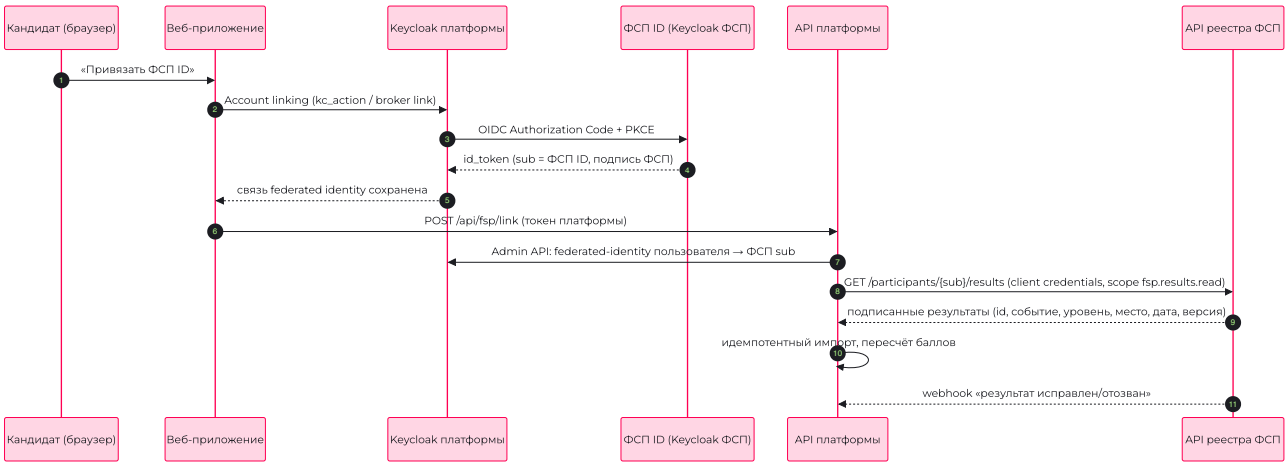
2. **Экспертный эталон.** 60–100 добровольцев, независимое структурированное интервью двумя экспертами вслепую, сравнение с грейдом теста (точность, взвешенная каппа), при необходимости — сдвиг порогов.
3. **Подбор.** Работодатели-эксперты размечают релевантность кандидатов по реальным потребностям, включая не попавших в выдачу. Метрики P@k и nDCG@k с базовыми линиями. Затем A/B-тест на доле принятых приглашений.
4. **Мониторинг.** Контрольные карты доли верных ответов при фиксированном θ (признак утечки), ротация вариантов, версия движка в каждой попытке.

Интеграция с реестром ФСП

Цель. Профиль кандидата получает достижения из реестра ФСП автоматически и только из доверенного источника. Набранный вручную номер участника ничего не доказывает, поэтому привязка всегда подтверждается владением учётной записью.

Как сделано в прототипе. Интеграция спрятана за интерфейсом `FspRegistry`. Реализация `MockFspRegistry` читает синтетический реестр (таблицы с участниками и результатами), который заполняется миграцией. Владение проверяется сравнением: e-mail участника в реестре должен совпасть с подтверждённым e-mail пользователя в Keycloak. После привязки результаты импортируются как подтверждённые достижения: событие, дисциплина, дата, место или роль, уровень. Баллы для ранжирования считаются по явной версионируемой политике. Отвязка удаляет импортированные записи и пересчитывает выдачу. Кандидат без истории ФСП проходит весь путь без штрафа: бонуса за достижения у него нет, а в объяснении выдачи стоит строка «История ФСП не привязана».

Целевая схема с ФСП ID на Keycloak.



- **Идентичность.** ФСП ID подключается к Keycloak платформы как Identity Provider (OIDC brokering). Привязка — стандартный account linking: пользователь входит в ФСП ID, и Keycloak сохраняет связь `federated identity`. Подменить чужой ФСП ID нельзя, потому что субъект берётся из подписанного `id_token` ФСП.
- **Данные.** Коннектор работает от сервисной учётной записи (client credentials) и запрашивает только результаты привязанного участника. Хранятся внешний ID результата, тип (участие, призёр, победитель), уровень соревнования, дата, источник и версия политики, статус `verified/revoked` и время последней сверки. Ключ уникальности — `(source, external_id, candidate_id)`, поэтому повторный импорт ничего не дублирует.
- **Актуальность.** Webhook реестра или периодическая сверка отзывают и исправляют результаты, после чего выдача пересчитывается. Отзыв согласия кандидатом удаляет импортированные данные.
- **Что увидит работодатель.** Только то, что кандидат разрешил в настройках приватности: название события, уровень, место и дату, без внутренних

идентификаторов ФСП.

- **Что нужно от ФСП.** Регистрация клиента OIDC в ФСП ID, контракт API результатов (поля, коды уровней, правила отзыва), тестовый стенд с тестовыми участниками и соглашение об обработке персональных данных.

Безопасность и 152-ФЗ

- **Аутентификация** через Keycloak 26: регистрация по e-mail с обязательным подтверждением адреса, защита от перебора паролей, пароли хранятся только в Keycloak в виде хешей Argon2. Браузер использует публичный клиент с Authorization Code + PKCE, без client secret. API проверяет подпись JWT по JWKS, а также `iss`, `aud=fsp-api` и срок действия.
- **Разграничение прав.** Роль (специалист или работодатель) выбирается один раз и хранится в БД. Каждый маршрут API проверяет роль и владение записью на сервере. Кандидат видит только свои приглашения и отклики. Работодатель не получает e-mail и телефон кандидата, пока кандидат не принял приглашение или сам не откликнулся на вакансию.
- **Согласия.** Согласие на обработку данных и согласие на публикацию профиля в подборках даются раздельно. Отзыв согласия удаляет профиль, ответы на тесты, черновики резюме, достижения и приглашения кандидата. Настройки приватности определяют, какие разделы профиля видит работодатель.
- **Минимизация.** Контакты не попадают в списки и объяснения выдачи. Ключи ответов на тесты хранятся только на сервере. В логах нет токенов и персональных данных.
- **Транспорт и инфраструктура.** Наружу открыт только Traefik: HTTPS с сертификатом Let’s Encrypt и редирект с HTTP. Базы данных и Mailpit доступны только внутри сети Docker. Секреты генерируются при первом развёртывании и лежат только на сервере, вне репозитория.
- **Для промышленной эксплуатации** нужны: хранение данных на территории РФ, назначенный ответственный за обработку ПДн, уведомление Роскомнадзора, журналы доступа, резервное копирование с шифрованием и регламент удаления по запросу субъекта.

Защита от фиктивных вакансий и недобросовестных работодателей (концепция)

В MVP не реализуется (так оговорено в ТЗ). Архитектура к этому готова: роль работодателя и все его действия (вакансии, приглашения, задания) уже привязаны к одной учётной записи компании.

Уровень	Механизм
Верификация компании	Проверка ИНН и ОГРН по ЕГРЮЛ (API ФНС или DaData), подтверждение домена (e-mail на корпоративном домене или DNS TXT-запись), подтверждение представителя через Госуслуги (ЕСИА) для расширенного статуса. Бейдж «Компания проверена» виден кандидату в приглашении.
Лимиты до верификации	Ограничение числа приглашений и вакансий в сутки и запрет массовых одинаковых приглашений. Вилка зарплаты обязательна и проверяется на правдоподобие по рынку для специализации и грейда; аномалии (например, junior за 1 млн ₽) уходят на модерацию.
Сигналы риска	Скоринг работодателя: доля отклонённых приглашений, жалобы кандидатов, приглашения без последующего общения, повторяющиеся тексты, подозрительные домены и контакты (мессенджеры вместо корпоративной почты), запрос оплаты или персональных данных сверх необходимого.
Обратная связь	Кнопка «Пожаловаться» в приглашении и вакансии. Несколько независимых жалоб автоматически скрывают вакансию до модерации. Рейтинг работодателя строится по подтверждённым взаимодействиям.
Защита кандидата	Контакты кандидата раскрываются только после его согласия: принятое приглашение или собственный отклик. Работодатель с низким рейтингом не видит кандидатов, у которых включён фильтр «только проверенные компании».
Модерация	Очередь модерации с правилами и ручной проверкой. Логи всех действий работодателя хранятся для разбора инцидентов.

Интеграция с ATS работодателей (концепция)

Цель — чтобы рекрутер работал в своей ATS (Huntflow, Potok, E-Staff, Talantix, SAP SuccessFactors и др.), а платформа поставляла в неё подтверждённых кандидатов.

- **Исходящие события (webhooks).** `invitation.accepted`, `application.created`, `application.status_changed`, `task.submitted`. Каждое событие подписано HMAC-SHA256, отправляется повторно с экспоненциальной задержкой и несёт ключ идемпотентности. Состав данных — только то, на что кандидат дал согласие.
- **API для ATS.** Сервисные ключи работодателя (OAuth2 client credentials) со scope `candidates.read`, `invitations.write`, `vacancies.write`. Через них можно выгрузить карточку кандидата (категория, результаты тестов, достижения ФСП, PDF-профиль) и синхронизировать вакансии и статусы в обе стороны.
- **Формат.** JSON по OpenAPI-схеме платформы с маппингом на модели популярных ATS. Плюс стандартизированный PDF-профиль, который уже генерирует платформа.
- **Коннекторы.** Модуль-адаптер на каждую ATS (маппинг полей и статусов) работает на общей очереди событий. Первым делом — Huntflow и Potok, у которых открытые API.
- **Безопасность.** Отзыв ключей, журнал выгрузок и отзыв согласия кандидатом с отправкой события `candidate.consent_revoked`. По нему ATS обязана удалить данные.

Описание API

Полная спецификация OpenAPI 3.0 с запросами, ответами, схемами и кодами ошибок: `apps/api/src/main/resources/openapi.yaml`. Развёрнутый экземпляр отдаёт её по адресу https://brokenbrain.online/openapi.yaml; её можно открыть в Swagger UI или Postman. Все маршруты `/api/*` требуют токен доступа Keycloak (`Authorization: Bearer …`) с `iss` реалма fsp-career и `aud=fsp-api`. Ролевые маршруты проверяют роль, выбранную через `POST /api/me/role`. Ошибки возвращаются в едином формате `{"error": "…"}` с кодами 400 (валидация), 401 (нет или неверный токен), 403 (не та роль или нет согласия), 404 (нет записи или чужая запись), 409 (недопустимый переход статуса, пересдача раньше срока, конфликт привязки), 500.

Служебные

Метод	Путь	Назначение	Ошибки
`GET`	`/health`	Public process liveness check	
`GET`	`/openapi.yaml`	Public OpenAPI 3 specification	

Аккаунт и роль

Метод	Путь	Назначение	Ошибки
`GET`	`/api/me`	Current user and the application role bound to the account (null before the first choice)	401
`POST`	`/api/me/role`	Choose the account role once (candidate or employer); repeating the same choice is idempotent	400, 401, 409

Профиль кандидата, резюме, приватность, ФСП

Метод	Путь	Назначение	Ошибки
`GET`	`/api/candidates/me`	Read own candidate profile (candidate role)	401, 403
`PUT`	`/api/candidates/me`	Update own profile with processingConsent true; publication consent is independent	400, 401, 403

Метод	Путь	Назначение	Ошибки
`DELETE`	`/api/candidates/me/consent`	Revoke processing and erase own profile, assessment answers, drafts, achievements and invitations	401, 403
`GET`	`/api/candidates/me/fsp`	Own FSP ID link status and imported results (candidate role)	401, 403
`POST`	`/api/candidates/me/fsp`	Link (or re-sync) an FSP participant ID and import its results as verified achievements (candidate role)	400, 401, 403, 404, 409
`DELETE`	`/api/candidates/me/fsp`	Unlink the FSP ID and delete imported FSP achievements (candidate role)	401, 403
`GET`	`/api/candidates/me/privacy`	Read own privacy settings (candidate role)	401, 403
`PUT`	`/api/candidates/me/privacy`	Replace own privacy settings (candidate role, processing consent required)	400, 401, 403
`POST`	`/api/candidates/me/resume-confirm`	Explicitly review/edit draft fields and save profile; verified email and publication consent stay unchanged	400, 401, 403, 404
`POST`	`/api/candidates/me/resume-upload`	Extract review-only suggestions from text-based PDF (candidate role, processing consent required)	400, 401, 403
`GET`	`/api/candidates/me/resume.pdf`	Download own standardized branded PDF resume (candidate role)	400, 401, 403

Тестирование

Метод	Путь	Назначение	Ошибки
`POST`	`/api/assessments/attempts`	Start an adaptive test for the declared grade (candidate role, processing consent required)	400, 401, 403, 404, 409
`GET`	`/api/assessments/attempts/current`	Open attempt with its current item, if any (candidate role)	401, 403
`POST`	`/api/assessments/attempts/{id}/answers`	Answer the current item; returns the next item or the final result (candidate role)	400, 401, 403, 404, 409
`POST`	`/api/assessments/blueprint`	Show how a vacancy becomes a test (any authenticated user)	400, 401, 404
`GET`	`/api/assessments/catalog`	Survey reference data, grades with cut-points, topics and demo vacancies (any authenticated user)	401
`GET`	`/api/assessments/eligibility`	Which grades the candidate may be tested for now (candidate role)	400, 401, 403
`GET`	`/api/assessments/history`	Own completed attempts with survey, target grade, θ, SE, score and confirmed grade, newest first (candidate role)	401, 403

Работодатель: компания и потребность

Метод	Путь	Назначение	Ошибки
`GET`	`/api/employers/company`	Read own employer company (employer role)	401, 403

Метод	Путь	Назначение	Ошибки
`PUT`	`/api/employers/company`	Update own employer company (employer role)	400, 401, 403
`GET`	`/api/employers/needs`	Read own hiring needs (employer role)	401, 403
`PUT`	`/api/employers/needs`	Update own hiring needs (employer role)	400, 401, 403

Подбор и банк кандидатов

Метод	Путь	Назначение	Ошибки
`GET`	`/api/bank/candidates`	Search the whole candidate bank by category, ordered by confirmed-profile strength (employer role)	400, 401, 403
`GET`	`/api/bank/candidates/{id}`	Employer-facing candidate card respecting the candidate's privacy settings (employer role)	400, 401, 403, 404
`GET`	`/api/matches`	Need-based selection ranked by relevance, with explanations and recommended categories (employer role)	400, 401, 403

Приглашения

Метод	Путь	Назначение	Ошибки
`GET`	`/api/invitations`	List own role's invitations (candidate or employer)	401, 403
`POST`	`/api/invitations`	Send invitation to published candidate with mandatory RUB salary range and a contact method (employer role)	400, 401, 403, 404

Метод	Путь	Назначение	Ошибки
`POST`	`/api/invitations/{id}/accept`	Invited candidate accepts sent/viewed invitation; contact disclosure begins	400, 401, 403, 404, 409
`POST`	`/api/invitations/{id}/reject`	Invited candidate rejects sent/viewed invitation	400, 401, 403, 404, 409
`POST`	`/api/invitations/{id}/view`	Invited candidate marks sent invitation viewed	400, 401, 403, 404, 409

Вакансии и отклики

Метод	Путь	Назначение	Ошибки
`GET`	`/api/candidates/me/applications`	Own applications with current status and vacancy conditions, newest first (candidate role)	401, 403
`DELETE`	`/api/candidates/me/applications/{id}`	Withdraw own sent/viewed application (candidate role); the employer loses access to it and to the contacts	400, 401, 403, 404, 409
`PUT`	`/api/employers/applications/{id}/status`	Change status of an application to own vacancy (employer role)	400, 401, 403, 404, 409
`GET`	`/api/employers/vacancies`	List own vacancies with application counters, newest first (employer role)	401, 403
`POST`	`/api/employers/vacancies`	Publish a vacancy with a mandatory RUB salary range (employer role)	400, 401, 403

Метод	Путь	Назначение	Ошибки
`PUT`	`/api/employers/vacancies/{id}`	Edit own vacancy (employer role); status is unchanged	400, 401, 403, 404
`GET`	`/api/employers/vacancies/{id}/applications`	Incoming applications to own vacancy with category, score, fit and contacts (employer role)	400, 401, 403, 404
`POST`	`/api/employers/vacancies/{id}/close`	Close own open vacancy (employer role); it disappears from the candidate list, applications stay	400, 401, 403, 404, 409
`POST`	`/api/employers/vacancies/{id}/reopen`	Reopen own closed vacancy (employer role)	400, 401, 403, 404, 409
`GET`	`/api/vacancies`	Browse open vacancies with company and salary range (candidate role)	400, 401, 403
`POST`	`/api/vacancies/{id}/applications`	Apply to an open vacancy (candidate role); discloses own contacts to that employer	400, 401, 403, 404, 409

Регулярные задания

Метод	Путь	Назначение	Ошибки
`GET`	`/api/candidates/me/task-submissions`	Own short-task answers with employer ratings, newest first (candidate role)	401, 403
`POST`	`/api/candidates/me/task-submissions`	Answer this week's task with a solution or a proposed approach (candidate role)	400, 401, 403, 409

Метод	Путь	Назначение	Ошибки
`GET`	`/api/candidates/me/tasks/current`	This week's short task (candidate role; processing consent and specialization required)	400, 401, 403
`PUT`	`/api/employers/task-submissions/{id}/rating`	Rate an answer to own task 1–5 with an optional comment; re-rating overwrites (employer role)	400, 401, 403, 404
`GET`	`/api/employers/tasks`	List own short tasks with answer counters (employer role)	401, 403
`POST`	`/api/employers/tasks`	Publish a short task for a specialization, optionally for one grade (employer role)	400, 401, 403
`POST`	`/api/employers/tasks/{id}/activate`	Return own task to rotation (employer role)	400, 401, 403, 404, 409
`POST`	`/api/employers/tasks/{id}/deactivate`	Remove own task from rotation (employer role); answers stay, open weekly offers are re-picked	400, 401, 403, 404, 409
`GET`	`/api/employers/tasks/{id}/submissions`	Answers to own task with candidate name and confirmed category, never contacts (employer role)	400, 401, 403, 404

Библиотеки и компоненты

Всё ПО открытое: лицензии MIT, Apache-2.0, BSD, PostgreSQL License. Код решения распространяется под лицензией MIT (`LICENSE`).

Инфраструктура

Компонент	Версия	Лицензия	Назначение
Docker Engine / Compose	29.x / v2.38+	Apache-2.0	контейнеризация, оркестрация стека
Traefik	3.7.13	MIT	обратный прокси, HTTPS (Let's Encrypt), маршрутизация по путям
PostgreSQL	16 (alpine)	PostgreSQL	данные платформы и отдельная БД Keycloak
Keycloak	26.0.8	Apache-2.0	аутентификация, регистрация с подтверждением e-mail, совместимость с ФСП ID
Mailpit	1.31.4	MIT	демо-почта для писем подтверждения
GitHub Actions (self-hosted runner)	2.337	MIT	CI/CD: деплой при каждом push в `main`

Backend (`apps/api`)

Библиотека	Версия	Назначение
Kotlin (JVM)	2.1.20	язык, JDK 21 (Eclipse Temurin)
Ktor Server (core, netty, auth, auth-jwt, content-negotiation, status-pages, cors)	3.1.2	HTTP API
kotlinx.serialization	(Kotlin 2.1.20)	JSON
JetBrains Exposed (core, jdbc, java-time, json)	0.61.0	SQL DSL, транзакции
Flyway	11.3.4	миграции схемы
HikariCP	6.2.1	пул соединений
PostgreSQL JDBC	42.7.5	драйвер БД
Auth0 java-jwt / jwks-rsa (через ktor-auth-jwt)	—	проверка подписи JWT по JWKS

Библиотека	Версия	Назначение
Apache PDFBox	3.0.4	разбор PDF-резюме и генерация PDF-профиля
Logback	1.5.18	логирование
Gradle	9.1	сборка

Frontend (`apps/web`)

Библиотека	Версия	Назначение
SvelteKit	2.70.3	веб-приложение, SSR, adapter-node 5.5.7
Svelte	5.57.1	UI (runes)
Vite	7.3.6	сборка
TypeScript	5.9.3	типизация
StyleX (+ unplugin)	0.19.1	атомарные стили
keycloak-js	26.2.4	вход по OIDC Authorization Code + PKCE
GSAP	3.15.0	анимации лендинга
@jis3r/icons (Moving Icons), @lucide/svelte	2.9.0, 1.52.0	иконки
Bun	1.4.2	пакетный менеджер и рантайм
svelte-check, Prettier	4.7.6, 3.9.9	проверки и форматирование

Шрифты и бренд

Шрифты Montserrat, Nunito и Play распространяются под лицензией SIL OFL и подключены локально. Знак и цвета взяты из брендбука ФСП.

Сборка, развёртывание и локальный запуск

Развёрнутый прототип

- Платформа: https://brokenbrain.online. Регистрация открыта; роль (специалист или работодатель) выбирается при первом входе.
- Письма подтверждения: демо-почта https://brokenbrain.online/mail/, если на сервере не настроен SMTP.
- Демо-аккаунты и тестовые данные описаны в разделе «Демонстрация» (см. «Обзор решения»).
- OpenAPI: https://brokenbrain.online/openapi.yaml.

Локальный запуск (Docker)

Нужны только Docker Engine и Compose v2.

```
git clone https://github.com/06ED/fsp-career && cd fsp-career
cp .env.example .env          # при необходимости поменяйте пароли
docker compose up --build -d  # web, api, PostgreSQL ×2, Keycloak, Mailpit
docker compose ps             # дождитесь статуса healthy (Keycloak стартует ~1 мин)
```

Сервис	Адрес
Платформа	http://localhost:3000
API и OpenAPI	http://localhost:8080, http://localhost:8080/openapi.yaml
Keycloak (админка)	http://localhost:8081/admin/ (логин и пароль из `.env`)
Почта (Mailpit)	http://localhost:8025

Сценарий: «Создать аккаунт» → подтвердить e-mail через ссылку в Mailpit → выбрать роль. Для работодателя удобно использовать второй аккаунт в окне инкогнито. Демо-данные загружаются скриптом `python3 docs/seed_demo.py` (см. README).

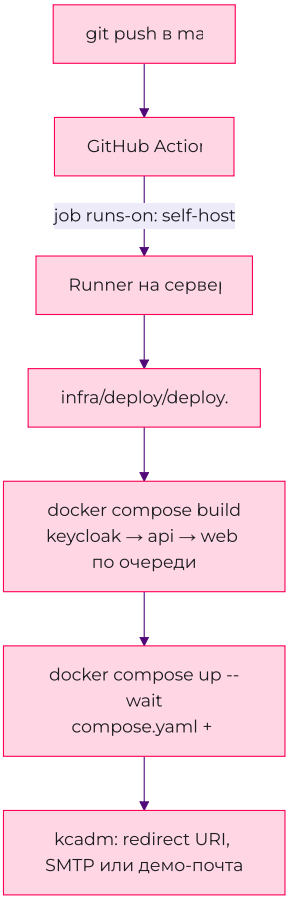
Порты публикуются только на `127.0.0.1`. Команда `docker compose down` сохраняет базы данных, `docker compose down -v` удаляет их без возможности восстановления.

Сборка без Docker (разработка)

- API: JDK 21 и Gradle 9: `cd apps/api && gradle test installDist`. Запуск: `gradle run` с переменными `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`, `KEYCLOAK_ISSUER`, `KEYCLOAK_AUDIENCE`.
- Web: Bun 1.4: `cd apps/web && bun install --frozen-lockfile && bun run dev` → http://localhost:5173. Проверки: `bun run check`, сборка: `bun run build`.

- Валидация механик (синтетический прогон): `cd apps/api && gradle validateAssessment validateMatching` пишет отчёты в `docs/validation/`.

Промышленное развёртывание (как устроено на brokenbrain.online)



1. GitHub Actions (`.github/workflows/deploy.yml`) при каждом push в `main` запускает задание на self-hosted runner, установленном на сервере. Задания не пересекаются: работает `concurrency`.
2. `infra/deploy/deploy.sh` при первом запуске генерирует секреты в `~/.config/fsp-career/.env`, вне репозитория. Затем по очереди собирает образы (на 4 ГБ RAM это важно): если сборка упала, работающая версия остаётся нетронутой. После этого поднимает стек с проверкой healthcheck.
3. `compose.prod.yaml` переводит Keycloak в production-режим за прокси (`/auth`), ограничивает heap JVM, публикует наружу только Traefik (80/443, Let's Encrypt) и задаёт маршруты: `/` — web, `/api` и `/openapi.yaml` — API, `/auth` — Keycloak, `/mail` — демо-почта.
4. Почта: если в env-файле есть `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`, скрипт настраивает их в Keycloak. Иначе письма идут в демо-почту, а сброс пароля отключён.

Требования к серверу: 2 vCPU, 4 ГБ RAM, swap 2 ГБ, 20 ГБ диска, Ubuntu 24.04 с Docker. Под нагрузкой в покое стек занимает около 1.5 ГБ RAM.

Ограничения и план развития

Ограничения прототипа

- **Данные синтетические.** Кандидаты, компании, результаты ФСП и метрики валидации — синтетика, помеченная «(демо)». Достижения ФСП приходят из синтетического реестра (`MockFspRegistry`) и не являются официальными.
- **Параметры заданий экспертные.** Калибровка на реальных ответах ещё не проводилась (план — в разделе «Валидация»). Концептуальные семейства с вариантами пока можно частично заучить.
- **Почта.** На демо-стенде без SMTP письма приходят в публичную демо-почту, а восстановление пароля отключено. Для эксплуатации нужен настоящий SMTP.
- **Масштаб.** Монолит рассчитан на десятки одновременных пользователей и тысячи кандидатов. Ранжирование считается в памяти по категории. Для сотен тысяч кандидатов нужен предрасчёт признаков и индексы.
- **Верификация работодателей** и модерация вакансий в MVP не делаются (по ТЗ). Концепция описана в разделе «Защита…».

План развития

Этап	Что делаем
1. Интеграция с ФСП (1–2 мес.)	ФСП ID как Identity Provider в Keycloak (account linking). Коннектор к API реестра результатов вместо mock. Webhook отзыва и исправления результатов. Согласование с ФСП политики баллов и формулировок.
2. Калибровка теста (2–3 мес.)	Пилот с участниками ФСП, калибровка IRT, экспертный эталон, расширение вариантов в концептуальных семействах, мониторинг утечек. Новые специализации: mobile, QA, DevOps, ML.
3. Качество подбора (параллельно)	Разметка релевантности работодателями, обучение весов на откликах «принял / отклонил» (learning-to-rank) с сохранением объяснимости, эмбеддинги для текстовой близости.
4. Доверие и безопасность	Верификация компаний (ИНН, ЕГРЮЛ, домен), скоринг риска работодателя, жалобы, модерация. Аудит ПДн и хранение в РФ.
5. Интеграции	Webhooks и API для ATS (Huntflow, Potok, E-Staff), уведомления в Telegram, экспорт PDF-профиля в ATS.
6. Продукт	Мобильная версия, рекомендации кандидату («что подтянуть до следующего грейда»), командные задания от работодателей, аналитика воронки для работодателя.